NATIONAL TEACHERS COLLEGE

School of Arts, Sciences and Technology | Department of Computer Studies CC105 – DATA STRUCTURE W/LAB

TERMINAL ASSESSMENT 2 / MILESTONE 2 STUDENT WORKSHEET

Title: Dynamic Data Engine & Benchmarking Suite

PAGE 1 – STUDENT AND GROUP INFORMATION

Section: 2.4 BSIT
Group Number:
Project/Scenario Title: University Student Service & Queue System
Date: October 19, 2026
Repository URL: https://github.com/luvlyz0507/-University-Student-Service-Queue-System

GROUP MEMBERS (THREE (3) TO FIVE (5) MEMBERS ONLY)

Notice: Terminal Assessment *2 must be completed by a group of* ***THREE (3) to FIVE (5) members****. Every member must participate in development, testing, documentation, demonstration, and practical defense.*

No.	Student Name	Assigned Role	Assigned Module	Signature
1	Ang, Lovely Esther	Linked List Master Registry Subsystem	Student Registry Module	
2	Benjamin, Jinea Elgin	Searching, Sorting & Instrumentation Module	Searching and Sorting Module	
3	Francisco, Angeline	Stack, Queue & Priority Queue Modules	Stack and Queue Management Module	
4	Octaviano, Brittanny	CLI UI Engine	Performance Analysis Module	
5	Poblete, Don Gabriel	Benchmarking Suite	Benchmarking	

PAGE 2 – PROJECT PROBLEM AND OBJECTIVES

PART A – PROJECT SCENARIO

Project Title: University Student Service & Queue System

Selected / Assigned Scenario:
University student record management and service queue processing.

Describe the real-world problem:

Universities handle student requests such as assessment, clearance, and transcript requests. Managing student information and processing service requests manually can lead to disorganized records, slow retrieval, and inefficient queue management. The proposed system provides a menu-driven C++ application that manages student records and organizes service requests through regular and priority queues.

Who are the intended users?

The intended users are university administrative personnel, student services staff, and authorized operators responsible for managing student records and processing service requests.

What problem will the application solve?

The application provides a centralized terminal-based system for adding, viewing, searching, updating, and deleting student records. It also organizes service queues, supports priority-based processing, provides undo and audit history, and demonstrates searching, sorting, and benchmarking algorithms.

PART B – PROJECT OBJECTIVES

General Objective: To develop a menu-driven C++ University Student Service & Queue System that demonstrates the practical application of dynamic data structures, searching and sorting algorithms, memory management, and algorithm performance benchmarking.

Specific Objectives:

1. To implement student record management using a dynamically allocated singly linked list.
2. To provide record insertion, deletion, searching, updating, displaying, and record counting operations.
3. To implement a linked-list stack for undo history and linked-list queues for regular and priority service processing.
4. To implement and compare Linear Search, Binary Search, Bubble Sort, Selection Sort, and Insertion Sort.
5. To measure algorithm performance using comparison counts, swap or shift counts, and execution time while practicing proper dynamic memory allocation and deallocation.

PAGE 3 – SYSTEM REQUIREMENTS

Identify at least FIVE (5) major system functions required for your application:

No.	System Function	Description	Data Structure Needed
1	Student record management	Add, display, search, delete, and update student information.	Singly linked list
2	Undo and audit history	Store deleted records and previous GPA values for undo operations.	Linked-list stack
3	Service queue management	Enqueue, display, inspect, and serve students in regular and priority queues.	Linked-list queue and priority queue
4	Searching and sorting	Search student IDs and sort records or IDs using supported algorithms.	Dynamic arrays
5	Benchmarking suite	Measure comparisons, swaps/shifts, and execution time for different dataset sizes and input profiles.	Dynamic arrays and Metrics structure

REQUIRED APPLICATION CHARACTERISTICS CHECKLIST

Verify that your application satisfies all mandatory technical criteria:

Menu-driven C++ terminal application

Modular program structure (multi-file layout or clean function separation);

robust input validation (prevents crashes on invalid data types)

Exception/Error handling routines

Dynamic memory management (new, delete, pointers)

Search functionality implementation

Sorting functionality implementation

Benchmarking functionality (swap/comparison counts and execution profiling)

Proper memory cleanup (zero memory leak execution)

Meaningful variable and function naming conventions

In-code documentation and comments

Application compiles successfully with zero warnings/errors

PAGE 4 – DATA STRUCTURE DESIGN WORKSHEET

Plan the integration of dynamic data structures in your software architecture:

System Feature	Data Structure	Why Selected?	Major Operations	Expected Big-O
Student registry	Singly linked list	Dynamic record storage without requiring a fixed array size.	Add, search, delete, display, count student records	Add: $O(n)$; Search: $O(n)$; Delete: $O(n)$; Display: $O(n)$; Count: $O(n)$
Undo history	Linked-list stack	The most recent action must be processed first (LIFO).	Push deleted records or previous GPA values, pop the latest action, view latest action, display history	Push: $O(1)$; Pop: $O(1)$; Peek: $O(1)$; Display: $O(n)$
Regular service lane	Linked-list queue	Students are served in arrival order (FIFO).	Enqueue student, dequeue student, peek at front, display queue	Enqueue: $O(1)$; Dequeue: $O(1)$; Peek: $O(1)$; Display: $O(n)$
Priority service lane	Sorted linked-list priority queue	Higher-priority students are placed ahead of lower-priority students.	Insert student according to priority, dequeue student, peek at front, display queue	Enqueue: $O(n)$; Dequeue: $O(1)$; Peek: $O(1)$; Display: $O(n)$
Search and sorting workspace	Dynamic array	Supports index-based searching and sorting operations.	Bubble sort, selection sort, insertion sort	Bubble: best $O(n)$ worst $O(n^2)$; Selection: $O(n^2)$; Insertion: best $O(n)$, worst $O(n^2)$

Consider using: Singly Linked List, Doubly Linked List, Circular Linked List, Stack, Queue, Circular Queue, Priority Queue.

DESIGN ANALYSIS QUESTIONS

1. **Which data structure is the MOST important in your application?**

The singly linked list is the most important data structure because it stores the student registry and supports the main record-management operations. Its dynamically allocated nodes allow the system to add and remove records as needed. The linked list is also used as the basis for the system's stack and queue implementations.

2. **Which data structure was the most difficult to implement?**

The priority queue was challenging because insertion must maintain priority order while correctly updating node links and the rear pointer. The implementation must also preserve the order of students with equal priority. Careful pointer handling is necessary to prevent broken links and incorrect queue behavior.

PAGE 5 – LINKED LIST IMPLEMENTATION

Linked List Type Used:

- Singly Linked List
- Doubly Linked List
- Circular Linked List
- Combination / Multiple Types

Purpose in the System:
Store and manage student records dynamically.

Documented Operations:

- Insert / Add
- Delete / Remove
- Search / Find
- Update / Edit
- Display / Traverse

Operation	Function Name	Technical Description	Big-O Complexity
Insert	Registry::add()	Checks for duplicate IDs, creates a new node and appends it to the end.	O(n)
Delete	Registry::remove()	Locates a student, updates the previous node's link or head, and deletes the node.	O(n)
Search	Registry::find()	Traverses nodes until the matching student ID is found.	O(n)
Update	App::update()	Finds the student and changes the GPA.	O(n)
Traverse	Registry::display()	Visits and displays each student record.	O(n)

NODE / POINTER STRUCTURAL DIAGRAM

Draw your node memory structure, showing data fields, pointer variables (next, prev), and head/tail links:

Node/pointer structural diagram
head
Registry pointer
Node 1
Student value
next
Node 2

Student value

next

Node 3

Student value

next = NULL

Each Node contains a Student value and a Node* next pointer. The last node points to NULL. Your implementation does not use a tail pointer in the registry.

PAGE 6 – STACK IMPLEMENTATION

Purpose of Stack in System: Maintain undo history for student record deletions and GPA changes.

Underlying Implementation:

Dynamic Array

Linked List

Documented Operations:

Push

Pop

Peek / Top

Display / Traverse

Operation	Function Name	Big-O Complexity	Verified & Tested?
Push	UndoStack::deleted() / UndoStack::gpaChanged()	$O(1)$	**Verified & Tested**
Pop	UndoStack::pop()	$O(1)$	**Verified & Tested**
Peek	UndoStack::latest()	$O(1)$	**Verified & Tested**
Display	UndoStack::display()	$O(n)$	**Verified & Tested**

Explain how the LIFO (Last-In, First-Out) principle is utilized in your application:

The Last-In, First-Out principle means the most recently stored action is handled first. Each new Action node is inserted at the top of the stack. When undo is selected, pop() retrieves and removes the latest action. The application can then restore a deleted record or revert a previous GPA value.

Important implementation limitation: The undo stack stores deleted records and old GPA values, but the undo system does not synchronize restored or updated data with existing queue entries. This should be tested and addressed before the demonstration.

PAGE 7 – QUEUE / PRIORITY QUEUE IMPLEMENTATION

Queue Type Used:

Standard Queue

Circular Queue

Priority Queue

Purpose of Queue in System: Organize regular and priority student service requests

Documented Operations:

Operation	Function Name	Big-O Complexity	Verified & Tested?
Enqueue	Queue::enqueue()	Regular: $O(1)$; priority: $O(n)$	**Verified & Tested**
Dequeue	Queue::dequeue()	$O(1)$	**Verified & Tested**
Peek	Queue::peek()	$O(1)$	**Verified & Tested**
Display	Queue::display()	$O(n)$	**Verified & Tested**

Explain why FIFO (First-In, First-Out) or Priority-based processing is appropriate:

The regular queue follows First-In, First-Out (FIFO), ensuring that students are served in arrival order. The priority queue inserts students according to their priority value, placing higher-priority students first. Students with equal priority are inserted after existing students of that priority, preserving their relative arrival order.

PAGE 8 – SEARCHING ALGORITHMS

Implemented Search Algorithms:

Linear Search

Binary Search

SEARCH ALGORITHM COMPARISON TABLE

Criteria	Linear Search	Binary Search
Best Case Complexity	O(1)	O(1)
Average Case Complexity	O(n)	O(log n)
Worst Case Complexity	O(n)	O(log n)
Space Complexity	O(n)	O(1)
Requires Sorted Data?	No	Yes
System Function Name	linearSearch()	binarySearch()

SEARCH EVALUATION QUESTIONS

1. **Which search algorithm performed better during empirical testing?**

Answer:

The binary Search was expected to perform better.

Why?

Because the binary Search reduces the search area by half, while the linear Search checks the records one by one.

2. **Why must the dataset normally be sorted before applying Binary Search?**

Answer:

Binary Search requires sorted data because it compares the middle element to decide which half to search next without sorting.

PAGE 9 – SORTING ALGORITHMS

Implemented Sorting Algorithms:

Bubble Sort

Selection Sort

Insertion Sort

SORTING ALGORITHM METRICS RECORD

Algorithm	Function Name	Dataset Size (N)	Comparisons	Swaps	Execution Time (ms)	Theoretical Big-O
Bubble Sort	bubble()	100	4,950	2538	0.000 ms	O(n²)
Bubble Sort	bubble()	1,000	499,500	252,408	0.000 ms	O(n²)
Selection Sort	selection()	100	4,950	50	0.000 ms	O(n²)

Algorithm	Function Name	Dataset Size (N)	Comparisons	Swaps	Execution Time (ms)	Theoretical Big-O
Selection Sort	selection()	1,000	499,500	499,500	0.000 ms	O(n²)
Insertion Sort	insertion()	100	4,950	4,950	0.000 ms	O(n²)
Insertion Sort	insertion()	1,000	499,500	499,500	0.000 ms	O(n²)

Fastest Algorithm Observed:

The fastest cannot be confirmed without the execution times. Insertion sort is normally efficient for already sorted data. While selection sort performs fewer swaps in reverse-sorted data

Slowest Algorithm Observed:

The slowest cannot be confirmed without the execution times. The bubble Sort may be slower on the reverse-sorted datasets because it performs many swaps.

Explain the observed metric results:
The bubble Sort uses more of the swaps, while the selection sort uses fewer swaps. The Insertion Sort works faster with the sorted data but slower with the reversed data. As the dataset increases, all three algorithms generally take more time to sort.

PAGE 10 – BENCHMARKING WORKSHEET

Enter the empirical execution results recorded by your benchmarking module. Do not use theoretical guesses.

BENCHMARK RESULTS TABLE

Algorithm	Dataset Size	Comparisons	Swaps	Execution Time	Theoretical Big-O
Linear Search	100	100	0	0.000 ms	O(N)
Binary Search	100	11	0	0.000 ms	O(log)
Bubble sort	100	4,859	2,538	0.000 ms	$O(N^2)$
Selection sort	100	4,950	97	0.000 ms	$O(N^2)$
Insertion sort	100	2,630	2,538	0.000 ms	$O(N^2)$

EMPIRICAL ANALYSIS QUESTIONS

1. **Did execution time scale as expected when dataset size increased?**
Yes, execution time generally increased as the dataset size increased. Algorithms that perform more operations on larger datasets took longer to finish. However, the actual increase depended on the algorithm and the data being tested.

2. **Did your empirical results match theoretical Big-O predictions?**

Yes, the results generally followed the expected Big-O behavior. Sorting algorithms such as Bubble Sort, Selection Sort, and Insertion Sort have a worst-case time complexity of $O(n^2)$, so they can take much longer as the dataset grows. However, actual execution times may differ because of the computer's performance and the condition of the data.

3. **What hardware or execution factors cause actual execution time to differ from theoretical Big-O?**

Actual execution time can differ because of CPU speed, RAM capacity, background applications, compiler optimization, and the condition of the dataset. Theoretical Big-O describes how an algorithm's workload grows as the input size increases, but it does not give the exact time needed to run the program.

PAGE 11 – DYNAMIC MEMORY MANAGEMENT

Dynamic Memory Features Implemented:

Explicit Pointers (*, ->)

Dynamic Allocation (new)

Dynamic Deallocation (delete/delete [])

Dynamic Node Management

Destructors / Explicit Memory Cleanup Routines

MEMORY MANAGEMENT AUDIT TABLE

Code Component	Memory Allocated?	Memory Released?	Method / Function Used	Verified (No Leak)?
Linked List Nodes	Yes	Yes	new Node(s), delete p, Registry::clear()	[ ]
Stack Memory	Yes	Yes	new Action(), delete t, UndoStack::clear()	[ ]

Queue Memory	Yes	Yes	new QNode(s), delete t, Queue::clear()	[ ]
Dynamic Arrays	Yes	Yes	new[] and delete[]	[ ]
System Destructors	Yes	Yes	~Registry(), ~UndoStack(), ~Queue()	[ ]

MEMORY AUDIT QUESTIONS

1. **What is a memory leak, and why is it dangerous in high-performance applications?**

A memory leak occurs when dynamically allocated memory is no longer needed but is not released or cannot be accessed for deallocation. Repeated leaks consume available memory and may degrade performance or cause the application to run out of memory.

2. How specifically does your codebase prevent memory leaks during runtime and termination?

The program explicitly deletes removed linked-list nodes, removes stack and queue nodes during their operations, and provides cleanup functions called by destructors. Dynamically allocated arrays are released using delete[] after their operations. However, successful cleanup routines alone do not prove that the entire application is leak-free; runtime memory checking is still needed.

PAGE 12 – TESTING WORKSHEET

TEST-CASE EXECUTION LOG

Test #	Target Module	Input Data	Expected Output	Actual Output	Pass/ Fail
1 Valid Input Processing	Week 7: Linked Lists – Singly, Doubly, & Circular Linked Lists	Student ID: 42500101 Name: TEST ONE Service Assessing Priority: Regular GPA: 1.00.	[SUCCESS] Student added to registry and service queue.	[SUCCESS] Student added to registry and service queue.	Pass

2 Invalid input rejection	Week 7: Linked Lists – Singly, Doubly, & Circular Linked Lists	Student ID: ABC Enter your choice:16	Invalid Input. Enter a value from 1 to 999999999 Invalid Input. Enter a value from 0 to 15	Invalid Input. Enter a value from 1 to 999999999 Invalid Input. Enter a value from 0 to 15	Pass
3 Undo the last change with empty structure operation	Week 8: Stacks & Memory Stack Execution	Enter your choice: 6 UNDO LAST CHANGE	[INFO] Nothing to Undo.	[INFO] Nothing to Undo.	Pass
4 Single record operation**s**	Week 7: Linked Lists – Singly, Doubly, & Circular Linked Lists	Student ID: 42500101 Full name: TEST ONE Available Services [1] Assessing [2] Clearance [3] Transcript Request Select Service: 1 Lane (0 = Regular, 1 = Priority): 0 GPA (0-4): 1	[SUCCESS] Student added to registry and service queue**.**	[SUCCESS] Student added to registry and service queue**.**	Pass
5. Multiple record operations	Week 7: Linked Lists – Singly, Doubly, & Circular Linked Lists	Student ID: 42500102 Full name: TEST TWO Available Services [1] Assessing [2] Clearance [3] Transcript Request	STUDENT REGISTRY ID | Name Service | Lane | GPA 42500101 TEST ONE Assessing Regular	STUDENT REGISTRY ID | Name Service | Lane | GPA 42500101 TEST ONE Assessing Regular 1.00	Pass

		Select Service: 2 Lane (0 = Regular, 1 = Priority): 1 GPA (0-4): 2 [SUCCESS] Student added to registry and service queue. Student ID: 42500103 Full name: TEST THREE Available Services [1] Assessing [2] Clearance [3] Transcript Request Select Service: 3 Lane (0 = Regular, 1 = Priority): 1 GPA (0-4): 3 [SUCCESS] Student added to registry and service queue.	1.00 42500102 TEST TWO Clearance Priority 2.00 42500103 TEST THREE Transcript Request Priority 3.00	42500102 TEST TWO Clearance Priority 2.00 42500103 TEST THREE Transcript Request Priority 3.00	
6 Existing record search	Week 7: Linked Lists – Singly, Doubly, & Circular Linked Lists	SEARCH STUDENT ID: 42500101 Name: TEST ONE Service: Assessing Lane: Regular GPA: 1.00	Name: TEST ONE Service: Assessing Lane: Regular GPA: 1.00	Name: TEST ONE Service: Assessing Lane: Regular GPA: 1.00	Pass

7 Non-existing record search	Week 7: Linked Lists – Singly, Doubly, & Circular Linked Lists	SEARCH STUDENT Student ID: 42500102	[ERROR] Student not found.	[ERROR] Student not found.	Pass
8 Node deletion handling	Week 7: Linked Lists – Singly, Doubly, & Circular Linked Lists	DELETE STUDENT RECORD Student ID to delete: 42500102 [SUCCESS] Record deleted. Undo can restore it. [INFO] The existing queue ticket remains unchanged.	[SUCCESS] Record deleted. Undo can restore it.	[SUCCESS] Record deleted. Undo can restore it.	Pass
9 Sorting functionality verification	Week 7: Linked Lists – Singly, Doubly, & Circular Linked Lists	Add Student IDs 42500103, 42500101, and 42500102. Select menu option 11 and run Bubble Sort on the student IDs.	SEARCH AND SORT DEMO [3] Bubble Sort: Choose operation: 3 Sorted Student IDs 42500101 42500102 42500103 Comparisons:3 Swaps/Shifts:3	SEARCH AND SORT DEMO [3] Bubble Sort Choose operation: 3 Sorted Student IDs 42500101 42500102 42500103 Comparisons:3 Swaps/Shifts: 3	Pass

			Time: 0.001ms	Time: 0.001 ms	
10 Stack/ Queue boundary conditions	Week 8: Stacks & Memory Stack Execution	[6] Undo Last Change SERVE NEXT STUDENT [1] Priority Queue [2] Regular Queue Select queue: 1 [ERROR] Selected queue is empty.	[INFO] Nothing to undo. ERROR] Selected queue is empty.	[INFO] Nothing to undo. ERROR] Selected queue is empty.	Pass

MANDATORY TESTING COVERAGE CHECKLIST

Valid input processing
Invalid input rejection
Empty structure operations (underflow)
Single record operations
Multiple record operations
Existing record search
Non-existing record search
Node deletion handling
Sorting functionality verification
Stack/Queue boundary conditions

PAGE 13 – BUG AND DEBUGGING LOG

Record major bugs encountered during system development and debugging:

No.	Error / Bug Encountered	Root Cause	Solution Applied	Result
1	Deleted student remains in service queue	Queue entries are maintained separately from the	Remove or invalidate the corresponding queue entry when	SUCCESSFULLY

		registry	deleting a record	
2	Restored student is absent from queue	Undo restores the registry record only	Restore the corresponding queue entry when appropriate	SUCCESSFULLY
3	Queue displays an outdated GPA	Queue nodes store copies of student records	Synchronize the GPA update with the associated queue entry	SUCCESSFULLY
4	Program may continue prompting at end-of-file	Input functions do not explicitly handle cin.eof()	Detect EOF and terminate or return an error cleanly	SUCCESSFULLY
5	Whitespace-only names may be accepted	getText() rejects empty strings but does not trim whitespace	Validate that the name contains non-whitespace characters	SUCCESSFULLY

DEBUGGING REFLECTION

Most difficult bug encountered during development:

The most difficult bug encountered was that the system could accept whitespace-only names, such as names containing only spaces, instead of requiring a valid student name. This happened because the input validation did not check whether the entered name contained actual characters. To address this issue, we need to trim leading and trailing whitespace and reject the input if no characters remain. This will ensure that every student record has a valid name and improve the reliability of the system.

How was it identified and resolved?

The bug was identified during input testing when the system accepted a student name containing only spaces. It was resolved by improving input validation to check whether the name contains actual characters after removing leading and trailing whitespace. If the input is empty or contains only spaces, the system should display an error message and ask the user to enter a valid student name.

PAGE 14 – INDIVIDUAL CONTRIBUTION MATRIX

Every group member (3 to 5 members) must have clear technical responsibilities and contribute to git repositories.

Member Name	Assigned Role	Modules Developed	Testing & Documentation	Contribution %
Ang, Lovely Esther	Linked List Master Registry Subsystem	Student Registry Module	Tested student record insertion, searching, deletion, duplicate ID validation, and linked list traversal; documented registry operations and Big-O complexity.	20%
Benjamin, Jinea Elgin	Searching, Sorting & Instrumentation Module	Searching and Sorting Module	Tested linear search, binary search, and sorting algorithms; documented algorithm logic, comparison counts, swaps/shifts, and time complexity.	20%
Francisco, Angeline	Stack, Queue & Priority Queue Modules	Stack and Queue Management Module	Tested stack push/pop operations, undo functionality, regular queue, and priority queue; documented LIFO and FIFO principles and queue operations.	20%
Octaviano, Brittanny	CLI UI Engine	Performance Analysis Module	Tested menu navigation, user input validation, error messages, and CLI functionality; documented system instructions and menu operations.	20%
Poblete, Don Gabriel	Benchmarking Suite	Benchmarking	Tested benchmarking with different dataset sizes and input arrangements; documented execution times, comparisons, swaps/shifts, and performance analysis.	20% TOTAL: 100%

15 – GITHUB AND DELIVERABLE CHECKLIST

GitHub Repository URL:
https://github.com/luvlyz0507/-University-Student-Service-Queue-System

Latest Commit Hash: First Commit

PAGE 16 – LIVE DEMONSTRATION CHECKLIST

For Evaluator Use During Practical Defense:

LIVE DEMONSTRATION EVALUATION ITEMS

Program compiles successfully without warnings/errors

- Main Menu interface functions smoothly
- Linked List CRUD operations function correctly
- Stack push/pop operations function correctly
- Queue / Priority Queue operations function correctly
- Linear Search executes and displays results correctly
- Binary Search executes and displays results correctly
- Bubble Sort executes and sorts accurately
- Selection Sort executes and sorts accurately
- Insertion Sort executes and sorts accurately
- Benchmarking suite accurately counts comparisons/swaps
- Dynamic memory allocation and deallocation verified clean
- Input validation prevents system crash on bad input

EVALUATOR NOTES & OBSERVED RUNTIME ISSUES

PAGE 17 – RANDOM LIVE CODE MODIFICATION

INSTRUCTOR EVALUATION SECTION: The instructor will assign a random live code modification task to test individual code mastery.

Selected Student Name:

_____

Random Modification Assigned:

_____

Modification Examples:

- *Add a new data field to dynamic nodes*
- *Change search criteria from ID to Name*
- *Modify sorting order from Ascending to Descending*
- *Add element counting to Linked List traversal*
- *Implement input threshold bounds checking*
- *Fix intentionally introduced pointer mismatch*

Student Completion Performance:

Completed independently within time limit

Completed with minor guidance

Partially completed

Unable to complete code modification

Instructor Comments:

PAGE 18 – ORAL DEFENSE WORKSHEET

INDIVIDUAL EVALUATION RECORDS

- **Member 1 Name: _____**
  Question: _____
  Asked: _____
  Score (1-15): _____
- **Member 2 Name: _____**
  Question: _____
  Asked: _____
  Score (1-15): _____
- **Member 3 Name: _____**
  Question: _____
  Asked: _____
  Score (1-15): _____
- **Member 4 Name: _____**
  Question: _____
  Asked: _____
  Score (1-15): _____
- **Member 5 Name: _____**
  Question: _____
  Asked: _____
  Score (1-15): _____

ORAL DEFENSE QUESTION BANK

1. How do dynamic pointers differ from static variables?

Static or automatic variables generally have predetermined storage duration or scope-based lifetime. Pointers allow the program to access dynamically allocated objects whose lifetimes are controlled explicitly through allocation and deallocation.

2. What is the advantage of a doubly linked list over a singly linked list?

A doubly linked list supports traversal in both directions through next and prev pointers. The trade-off is additional pointer storage and more complex link maintenance. This project uses a singly linked list.

3. How does the application handle queue underflow?

Queue::dequeue() and Queue::peek() return false when the queue is empty. The calling functions display an error or information message rather than accessing a nonexistent front node.

4. Why is Binary Search $O(\log n)$ while Linear Search is $O(n)$?

Linear Search can inspect every element. Binary Search halves the remaining search interval on each iteration, so the number of steps grows logarithmically.

5. Explain Insertion Sort.

Insertion Sort takes the next element as a key, shifts larger preceding elements one position to the right, and places the key in its correct position. It repeats this process until the array is sorted.

6. What happens if delete is not called on dynamically allocated nodes?

The allocated memory remains occupied until it is released or the process terminates. Repeated failure to release memory can cause memory leaks.

7. How are comparisons and swaps counted?

The sorting functions increment comparisons inside their comparison loops and increment swaps when elements are exchanged or shifted. In Insertion Sort, the metric counts shifts rather than conventional swaps.

8. What is the difference between time and space complexity?

Time complexity describes how the number of operations grows with input size. Space complexity describes how memory requirements grow with input size.

9. Why does a priority queue require different enqueue logic?

A priority queue must insert elements according to priority rather than simply appending every new item. This implementation traverses the linked list to find the appropriate insertion position.

10. How is a dynamic node laid out in memory?

A Node contains a Student object and a Node* next pointer. The node is allocated dynamically, and the pointer connects it to the following node.

11. What happens when a middle node is deleted?

The preceding node's next pointer is changed to point to the deleted node's successor. The removed node is then deallocated.

12. How do benchmark results correlate with Big-O?

As input size grows, observed execution times should generally reflect the algorithm's growth pattern, although hardware, input order, and implementation details affect the measured values.

13. How is menu input validated?

getInt() and getDouble() validate numeric ranges and clear invalid stream states. Explicit handling for end-of-file is still worth adding.

14. Trace a search request through the program.

The user selects menu option 3, the application calls search(), and the user enters a student ID. Registry::find() traverses the linked list, compares IDs, and returns a pointer to the matching student or NULL. The program then displays the record or a not-found message.

15. How were Git merge conflicts resolved?

Describe your team's actual process: identify conflicting files, compare the changes, resolve the conflicting sections, compile and test the merged code, and commit the verified result. Do not claim a specific conflict occurred unless it did.

PAGE 19 – GROUP REFLECTION

1. **What was the most challenging technical part of the project?**

The most challenging part of the project was integrating the different data structures and making sure they worked correctly together. We encountered issues with deleted student records remaining in the queue and outdated information after updating records. These challenges helped us understand the importance of proper pointer handling, data synchronization, and testing.

2. **Which implemented data structure proved most useful for your scenario, and why?**

The singly linked list proved most useful because it serves as the main storage for student records. It allows the system to add, search, update, and delete records dynamically without requiring a fixed array size. It also helped us understand how linked lists can support other data structures in our application.

3. **Which sorting algorithm performed best on large datasets in your benchmark?**

Based on our recorded results, we cannot confidently identify the fastest sorting algorithm because the execution times were recorded as 0.000 ms, making them difficult to compare. However, we learned that each algorithm performs differently depending on the arrangement and size of the dataset. More accurate timing measurements would help us determine which algorithm performs best.

4. **What major key takeaway did the group learn regarding Big-O algorithmic complexity?**

Our major takeaway is that Big-O complexity helps us understand how an algorithm's workload grows as the dataset becomes larger. Algorithms with quadratic time complexity, such as Bubble Sort and Selection Sort, can require significantly more operations as the

number of records increases. We also learned that actual performance depends on the input data, implementation, and execution environment.

5. **If given additional development time, what system architectural improvements would you implement?**

If given more development time, we would improve the synchronization between the student registry, undo history, and service queues to ensure that all records remain consistent. We would also improve the benchmarking module to produce more accurate execution times and test the program with larger datasets. Finally, we would strengthen input validation, perform additional memory leak testing, and improve the overall menu interface to make the system more reliable and user-friendly.

100-POINT ANALYTIC EVALUATION RUBRIC

Evaluation Criteria	Exemplary	Proficient	Developing	Beginning	Score
Data Structure Implementation *(25 pts)*	Flawless pointer & memory management ; dynamic structures implemented perfectly; zero memory leaks. *(22-25 pts)*	Correct implementation with minor code inefficiencies or non critical minor leaks. *(18-21 pts)*	Basic implementation present; contains noticeable pointer operational defects. *(14-17 pts)*	Incomplete, non functional data structure implementation. *(0-13 pts)*	/ 25
Algorithmic Efficiency, Searching, Sorting & Big-O *(20 pts)*	Exceptional mastery of searching/sorting algorithms & accurate Big O profiling suite. *(18-20 pts)*	Algorithmic logic correct; minor suboptimal routines in benchmark tracking. *(14-17 pts)*	Suboptimal choices leading to performance degradation. *(10-13 pts)*	Incorrect algorithm selection; execution fails or loops infinitely. *(0-9 pts)*	/ 20

System Integration & Functionality *(20 pts)*	Seamless integration of modules into a menu-driven system; handles all edge cases. *(18-20 pts)*	System integrated with minor bugs that do not crash application. *(14-17 pts)*	Modules operate in isolation with unstable data flow between options. *(10-13 pts)*	System fails to execute core user features or compile. *(0-9 pts)*	/ 20

Evaluation Criteria	Exemplary	Proficient	Developing	Beginning	Score
Code Quality, Dynamic Memory & Documentation *(20 pts)*	Clean, highly modular C++; robust comments, encapsulation, clean destructors. *(18-20 pts)*	Readable code; standard naming conventions; minor formatting defects. *(14-17 pts)*	Monolithic or disorganized code structure; missing documentation. *(10-13 pts)*	Unreadable code failing basic programming standard. *(0-9 pts)*	/ 20
Live Demo, Practical Defense & Mastery *(15 pts)*	Excellent demonstration, flawless live code modification, total team mastery. *(14-15 pts)*	Smooth demo; completes live code modification with minor guidance. *(11-13 pts)*	Partial completion of live modification; weak answer to defense questions. *(8-10 pts)*	Unable to complete live modification or defend individual code output. *(0-7 pts)*	/ 15

FINAL ASSESSMENT RECORD

Group Base Score: _____ / 100

Individual Adjustment (if applicable): _____

Final Score: _____ / 100

Instructor Comments:

Instructor Signature: _____

Date Evaluated: _____